Chapter 23 - Memory Model and MMIO Map

Part IV of this manual is the bridge between BASIC and machine language. From here on, every chapter assumes you want to read and write the Intuition Engine's hardware directly: from BASIC with `PEEK` and `POKE`, from the machine monitor, or from a program written in one of the six processor languages described in Chapters 24 through 29.

Everything you do at that level happens through one shared address space. This chapter is the map of that space.

23.1 The address space

Intuition Engine has a single 32-bit physical address space, `4` gigabytes in size, addressed from `0x00000000` to `0xFFFFFFFF`. Every device, every chunk of RAM, every register lives somewhere in that space. There are no segments, no protection rings, no protection layers above the address itself. A `MOVE.L D0, ($F0700).L` from M68K and a `STORE.L (R5), R6` from IE64 with `R5 = 0xF0700` arrive at the same byte. The 6502 and Z80 reach the same `32`-bit register through the small set of bus-translation apertures described in §23.9, with the caveat that some 16-bit addresses are captured by their CPU-adapter intercepts before the bus sees them.

This is not how every processor sees the world. The 6502 and the Z80 can address only `64` kilobytes directly. Each of those CPUs has a small set of apertures into the larger address space; the apertures are described in the per-CPU chapters. The 68K, x86, IE32 and IE64 all see the full address space without a window.

23.1.1 Three zones

The space divides into three zones:

Range	Size	Contents
`0x00000000` – `0x0009EFFF`	`636K`	Main RAM (low memory, code and data)
`0x0009F000` – `0x000FFFFF`	`388K`	Stack, VRAM apertures and MMIO
`0x00100000` – `0x005FFFFF`	`5M`	Main VRAM
`0x00600000` – `0xFFFEFFFF`		Extended RAM (sized at boot)
`0xFFFF0000` – `0xFFFFFFFF`	`64K`	Sign-extended alias of `0x00000000`–`0x0000FFFF`

The `5`-megabyte VRAM block from `0x100000` to `0x5FFFFF` is wired directly to the VideoChip framebuffer (Chapter 4). Writes here appear on screen the next time the compositor refreshes.

Memory beyond `0x600000` is plain RAM. Its upper bound depends on the amount of RAM the Intuition Engine was started with. The two words at `0xF2400`/`0xF2404` (`SYSINFO_TOTAL_RAM_LO`/`HI`) report the byte size of total guest RAM; `0xF2408`/`0xF240C` report the size visible to the currently executing CPU profile.

23.1.2 The sign-extended alias

The top `64` kilobytes (`0xFFFF0000` – `0xFFFFFFFF`) are not real memory of their own. They are a mirror of the bottom `64` kilobytes: address `0xFFFFFF00` reads the same byte as `0x0000FF00`.

This alias exists for the 68K. A pre-decrement from a register holding zero (`-(SP)` with `SP = 0`) wraps to `0xFFFFFFFC`, which must still hit the stack. The alias makes that work.

23.2 Main RAM

Main RAM begins at `0x00000000` and runs up to the bottom of the I/O region.

```
0x00000000  Vector table     (interrupt vectors and reserved slots)
0x00001000  Program code     (PROG_START)
0x00002000  Stack region     (grows downward from STACK_START)
0x0009F000  Top of stack     (STACK_START - initial SP for IE32)
0x0009FFFF  End of low RAM
```

The vector table layout depends on which CPU is active. The 68K follows the standard 68000 vector table (Chapter 28). The 6502 keeps its `NMI`/`RESET`/`IRQ` vectors in the top six bytes of the 16-bit page that maps to `0x0000FFFA`–`0x0000FFFF`. IE64 and IE32 use their own vector formats (Chapters 24 and 25).

`PROG_START` at `0x1000` is the address where the image executor (Chapter 31) loads program images, and is the default `RUN` entry point. Programs are free to extend past `0x1000` upward and to use the stack region below it for data, but the floor at `0x2000` is the conventional ceiling for the stack and the floor for static data. Programs that need more stack than `637` kilobytes can simply place `SP` higher in extended RAM.

23.3 The MMIO region

Everything between `0xF0000` and `0xFFFFF`, together with the two VGA legacy VRAM apertures at `0xA0000` and `0xB8000` and the ULA VRAM aperture at `0xFA000`, is memory-mapped I/O: the registers of the six picture chips, the ten audio engines, the file system bridge, the coprocessor, and various small control surfaces. A read or write to an MMIO address does not touch RAM; it asks the corresponding device to do something.

The full map:

Range	Size	Device
`0xA0000`–`0xAFFFF`	`64K`	VGA graphics VRAM (mode `0x12`/`0x13`/`0x14`)
`0xB8000`–`0xBFFFF`	`32K`	VGA text VRAM
`0xF0000`–`0xF049B`	`1180B`	VideoChip + palette + extended blitter
`0xF0700`–`0xF07FF`	`256B`	Terminal, mouse, keyboard, RTC
`0xF0800`–`0xF0B7F`	`896B`	SoundChip (core six engines)
`0xF0BC0`–`0xF0BD7`	`24B`	MOD player
`0xF0BD8`–`0xF0BF3`	`28B`	WAV sample player
`0xF0C00`–`0xF0C20`	`33B`	PSG (AY-3-8910/YM2149)
`0xF0C40`–`0xF0CFF`	`192B`	SID2 flex channels (4–6)
`0xF0D00`–`0xF0D20`	`33B`	POKEY
`0xF0D40`–`0xF0DFF`	`192B`	SID3 flex channels (7–9)
`0xF0E00`–`0xF0E2D`	`46B`	SID (6581/8580)
`0xF0E80`–`0xF0EFF`	`128B`	SFX trigger block
`0xF0F00`–`0xF0F1F`	`32B`	TED audio
`0xF0F20`–`0xF0F5F`	`64B`	TED video

Range	Size	Device
0xF1000–0xF13FF	1K	VGA registers
0xF1400–0xF140F	16B	HOST helper MMIO
0xF2000–0xF2017	24B	ULA registers
0xF2100–0xF213F	64B	ANTIC
0xF2140–0xF21B7	120B	GTIA
0xF2200–0xF221F	32B	File I/O
0xF2260–0xF22AF	80B	Paula DMA (audio)
0xF2300–0xF231F	32B	Media loader
0xF2320–0xF233F	32B	Image executor
0xF2340–0xF238F	80B	Coprocessor control
0xF2390–0xF23AF	32B	Clipboard bridge
0xF23B0–0xF23BF	16B	Coprocessor extended monitor
0xF23C0–0xF23DF	32B	IRQ diagnostics
0xF23E0–0xF23FF	32B	Bootstrap file bridge
0xF2400–0xF24FF	256B	SysInfo (RAM-size discovery)
0xF8000–0xF87FF	2K	Voodoo 3D registers
0xFA000–0xFBAFF	6912B	ULA VRAM (bitmap + attrs)

A few small ranges between these blocks read as zero on read and ignore on write. They are reserved for future devices.

23.3.1 Width and alignment

Every MMIO register is 32 bits wide. Most accept a 32-bit read or write; a few accept 8-bit or 16-bit accesses for the benefit of the 6502 and Z80. Where width matters it is noted in the chip's chapter.

64-bit accesses to a legacy 32-bit MMIO region split into two 32-bit operations (low half first, then high half) when the bus's 64-bit legacy policy is set to "split". They fault by default.

Multi-byte registers are little-endian: a 32-bit write of 0x12345678 to 0xF0800 puts 0x78 at 0xF0800, 0x56 at 0xF0801, 0x34 at 0xF0802, 0x12 at 0xF0803. This is true even when the executing CPU is M68K, which is big-endian for plain RAM accesses; the bus normalises every MMIO access to little-endian before delivering it to the device.

23.4 The terminal region

0xF0700–0xF07FF holds the basic human-facing peripherals: the text terminal, mouse, keyboard scancodes, and a real-time clock.

Address	Name	Description
0xF0700	TERM_OUT	Write a byte: emit to terminal
0xF0704	TERM_STATUS	b0 input ready, b1 output ready
0xF0708	TERM_IN	Read next input byte (dequeues)

Address	Name	Description
0xF070C	TERM_LINE_STATUS	b0 complete line available
0xF0710	TERM_ECHO	b0 local echo enable (default 1)
0xF0724	TERM_CTRL	b0 line input mode
0xF0728	TERM_KEY_IN	Read next raw key
0xF072C	TERM_KEY_STATUS	b0 raw key available
0xF0730	MOUSE_X	Absolute X position
0xF0734	MOUSE_Y	Absolute Y position
0xF0738	MOUSE_BUTTONS	b0 left, b1 right, b2 middle
0xF073C	MOUSE_STATUS	b0 changed-since-last-read
0xF074C	MOUSE_CTRL	b0 request relative/captured mode
0xF0740	SCAN_CODE	Dequeue raw keyboard scancode
0xF0744	SCAN_STATUS	b0 scancode available
0xF0748	SCAN_MODIFIERS	b0 shift, b1 ctrl, b2 alt, b3 caps
0xF0750	RTC_EPOCH	UTC seconds since 1970-01-01
0xF0754	MOUSE_DX	Signed relative X delta (clears)
0xF0758	MOUSE_DY	Signed relative Y delta (clears)
0xF07F0	TERM_SENTINEL	Write 0xDEAD to halt the CPU

The M68K reaches TERM_OUT through a second alias as well:

- TERM_OUT_SIGNEXT = 0xFFFFF700: the sign-extended .W form that an M68K MOVE.B D0, ($F700).W resolves to. The bus folds this back onto the same 0xF0700 register through the sign-extended mirror described in §23.1.2.

The 6502 and Z80 have no equivalent 16-bit alias for TERM_OUT: the only 16-bit address that would translate to 0xF0700 is 0xF700, which the CPU adapters intercept as BANK1_REG_LO before the bus translation runs. Bare 6502 or Z80 code that needs to emit text uses the ULA, the VGA, or the SoundChip beeper instead. See Chapters 26 and 27 for the per-CPU details.

Chapter 36 covers the keyboard, mouse, and controller MMIO in detail. Chapter 37 covers the serial interface that overlays TERM_OUT/TERM_IN.

23.5 The system-information block

0xF2400–0xF24FF. Four read-only words let a program discover how much memory it has to play with:

Address	Name	Description
0xF2400	SYSINFO_TOTAL_RAM_LO	Low 32 bits of total RAM, bytes
0xF2404	SYSINFO_TOTAL_RAM_HI	High 32 bits of total RAM
0xF2408	SYSINFO_ACTIVE_RAM_LO	Low 32 bits of RAM visible to the active CPU
0xF240C	SYSINFO_ACTIVE_RAM_HI	High 32 bits of CPU-visible RAM

The total and active values can differ when a 16-bit profile (6502 or Z80) is the active CPU: total reports the physical RAM, active reports the window the small CPU can currently see.

23.6 The bootstrap file bridge

0xF23E0–0xF23FF. This eight-register block exposes the file system to bootstrap code that runs before BASIC is up.

Offset	Name	Direction	Description
+0	CMD	W	Command code, 0–7
+4	ARG1	W	First argument
+8	ARG2	W	Second argument
+12	ARG3	W	Third argument
+16	ARG4	W	Fourth argument
+20	RES1	R	First result
+24	RES2	R	Second result
+28	ERR	R	Error code (0 = success)

Command codes are 0 DISCOVER, 1 OPEN, 2 READ, 3 CLOSE, 4 STAT, 5 READDIR, 6 CREATE_WRITE, 7 WRITE. Chapter 34 has the full protocol.

Most programs never touch this region directly. BASIC's LOAD, SAVE, BLOAD and DIR go through it, as does the program executor. It is here for the rare case where you are writing your own loader.

23.7 The IRQ diagnostics block

0xF23C0–0xF23DF. Eight read-only words that report interrupt activity. These are diagnostic taps; ordinary programs do not need them.

Address	Name	Reports
0xF23C0	IRQ_DIAG_ISR	Interrupt-in-service bitmask
0xF23C4	IRQ_DIAG_FLAGS	b0 stopped, b1 in-exception, b2 INTENA, b3 running
0xF23C8	IRQ_DIAG_PENDING	Pending interrupt bitmask
0xF23CC	IRQ_DIAG_COUNTERS	L5 delivered (lo16) + L4 delivered (hi16)
0xF23D0	IRQ_DIAG_BLOCKED	L5 blocked (lo16) + L4 blocked (hi16)
0xF23D4	IRQ_DIAG_RTE	RTE count
0xF23D8	IRQ_DIAG_STOP_SPINS	Consecutive STOP iterations
0xF23DC	IRQ_DIAG_WATCHDOG	Latched watchdog event count

Chapter 30 describes the trap and exception model that backs these counters.

23.8 PEEK and POKE from BASIC

BASIC has two pairs of memory primitives:

- `PEEK(addr)` reads a 32-bit unsigned value. `POKE addr, value` writes a 32-bit unsigned value. Both require `addr` to be a multiple of 4; an unaligned address raises `?FC ERROR`.
- `PEEK8(addr)` reads a 8-bit unsigned value. `POKE8 addr, value` writes one byte. These accept any address and are the right choice for byte-oriented MMIO registers such as TED audio (`0xF0F00-0xF0F05`), the WAV volume bytes (`0xF0BF1-0xF0BF2`), the SID register file, and any single byte of the VGA palette.

```
10 REM read the VBlank flag from VideoChip
20 V = PEEK(&HF0008)
30 IF (V AND 1) = 0 THEN GOTO 20
40 REM ... do something at the start of VBlank
```

```
10 REM emit a banner one byte at a time
20 B$ = "INTUITION ENGINE"
30 FOR I = 1 TO LEN(B$)
40   POKE8 &HF0700, ASC(MID$(B$, I, 1))
50 NEXT I
60 POKE8 &HF0700, 13
```

The MMIO map decides what a `PEEK` actually returns. `0xF0700` is the `TERM_OUT` register: it is write-only and reads back zero. To poll terminal status, read `TERM_STATUS` at `0xF0704` (bit `0` input available, bit `1` output ready). Tables in the following chapters mark which MMIO addresses are write-only and which return a stable value on read.

23.9 Addresses on the smaller CPUs

The 6502 and Z80 cannot see more than 64 kilobytes at a time. For them, the I/O region appears at the top of the 16-bit space:

- 6502: addresses `0xF000-0xFFF9` map to `0xF0000-0xF0FF9`, except for the bank-register page `0xF700-0xF705` and the `VRAM_BANK_REG` at `0xF7F0`, which the CPU adapter intercepts. `0xFFFA-0xFFFF` is the 6502 vector table and is not translated. The VGA is reached at `0xD700-0xD70A`; the ULA at `0xD800-0xD817` with a paged VRAM port. Chapter 26 lists the full set of aliases.
- Z80: same `0xF000-0xFFF9` MMIO window with the same `$F700` bank-register intercept. The Z80 also exposes the VGA through `OUT (0xA0..0xAA)` port I/O, and the ULA through ports `0xFE`/`0xFD`/`0xBE`/`0xFA-0xFC` with a paged VRAM port.

The M68K, x86, IE32 and IE64 see the full 32-bit address space directly: a `POKE` at `0xF0700` from BASIC and a 68K `MOVE.L D0, ($F0700).L` reach the same register.

23.10 What comes next

Chapter 24 begins the per-CPU section with IE64, the native processor of the Intuition Engine. It is the easiest CPU to write for because it has the most general instruction set and the most registers; everything else in Part IV is the same story told through a smaller instruction set.